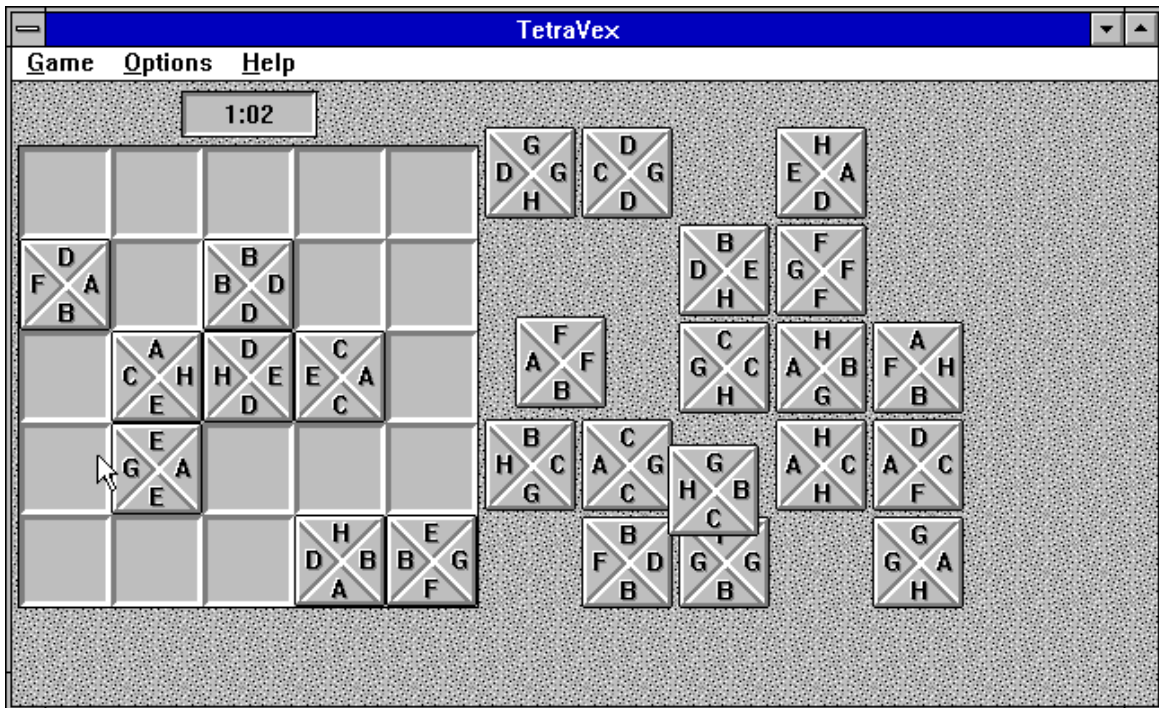


Parallelization of TetraVex Puzzle Solvers using Shared and Distributed Memory



Contents

Introduction	2
Problem Description	2
Solution Approach	3
Input and Puzzle Configurations	3
Data Structures	4
Serial and Shared Memory Approach	5
Distributed Memory Approach	6
Load Rebalancing with Distributed Memory	7
Performance and Analysis	8
7x7 TetraVex Puzzle with 9 Colors.....	8
Puzzle with Very Few Colors.....	9
Very Large Framed Puzzle with 23 Colors.....	10
Even Larger Framed Puzzle with 23 Colors.....	11
Further Work	12
Conclusion	12
References	14

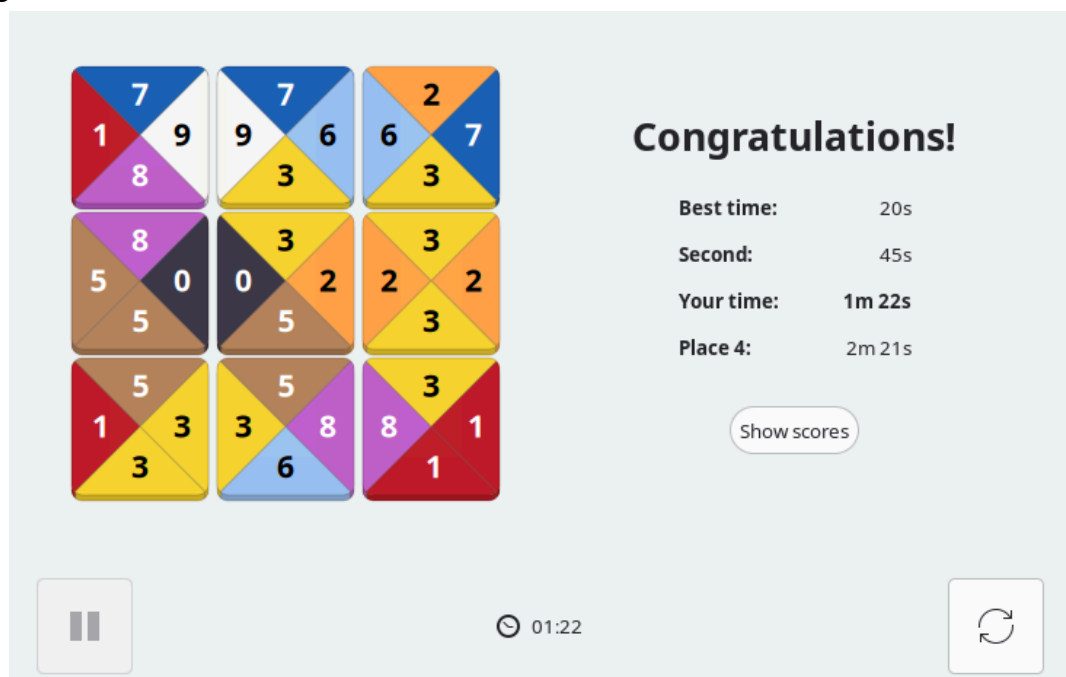
Computational puzzles such as Sudoku, Minesweeper, and TetraVex offer fascinating challenges in the fields of algorithm design and parallel computing. These types of puzzles are NP Complete and often grow exponentially in complexity with problem size. TetraVex in particular highlights this complexity with its factorial scaling of board configurations.

This project aims to compare parallel algorithms for solving TetraVex puzzles, specifically involving shared and distributed memory. Furthermore, increasing scalability using load rebalancing is analyzed, alongside its effects on communication overhead.

The following sections describe the TetraVex problem, the designs of the solvers, and the performance of different parallelization techniques across a variety of benchmark puzzles. The paper concludes with insights into which techniques work better for different puzzle configurations alongside suggestions for further improvement.

Problem Description

TetraVex is an edge matching puzzle game, where the player is given an n by n empty grid, and n^2 square pieces that can be placed in that grid. A single TetraVex puzzle piece is divided into 4 triangular edges, with a color or symbol on each. These pieces may not be rotated. The object of the puzzle is to arrange the pieces in such a way where every interior edge matches, leaving only the border edges unmatched. A version of the game was released in 1991 in Microsoft Entertainment Pack 3, which is a collection of games and puzzles made for Windows 3.1. The game is also available on Linux as an optional part of the GNOME desktop, or as an optional package.



GNOME TetraVex

TetraVex is a deceptively complex game. While a 2x2 or 3x3 game board might be easy to solve, the difficulty increases rapidly as the board size increases. This is due to the fact that the number of board configurations increases at a rate of $(n^2)!$, with only one or a few configurations being valid solutions. The puzzle is NP Complete, as it has been reduced to a variation of the SAT problem^[1]. As a result, there is no known polynomial time solution to TetraVex. Adding rotation to the pieces would make the puzzle even more difficult. An example of that is the Eternity II puzzle, designed by Christopher Monckton. After its release in July 2007, Monckton promised a prize of \$2 million for a solution before the end of 2010. The prize went unclaimed, and to this day, Eternity II remains unsolved, with the best known solution containing 10 unmatched edges, found in 2021 by Jonah Blackwood^[2]. In December 2024, a second solution with 10 unmatched edges was found, again using Blackwood's solver^[3].

Solution Approach

TetraVex is, at its core, similar to other board games such as Chess. In Chess, each board state has several move possibilities, and those moves cause the game to branch in different ways. Both can be represented as traversable game trees. Unlike Chess, however, it is impossible to “recover” from a bad move in TetraVex, but usually there are less moves that need to be considered, plus there is no uncertainty arising from another player. My first attempt at a TetraVex solver was written in Python last summer, and used a breadth-first backtracking search in order to find solutions. Additionally, that solver used a variety of pruning functions to reduce branching. For example, the solver would, on any given board state, only consider placing tiles on the space with the fewest possible options. That space would need to be filled later by something at some point, so selecting the most restricted space ensures the lowest possible branching factor. This solver was not parallelized, and as a result, it took a long time to solve puzzles with sizes around 7x7 with 10 distinct edge colors.

In this project, I will be comparing the performance of a serial TetraVex solver written in C++, a version that was parallelized with shared memory using OpenMP, and a version that was parallelized with distributed memory using MPI. Furthermore, I will be implementing periodic load rebalancing with MPI to reduce idle processors on large puzzles.

Input and Puzzle Configurations

The input to the solver takes the form of an array of pieces scrambled in no particular order. These pieces were generated by first creating a random solved puzzle from a seed, then shuffling the pieces around randomly and flattening them into an array of `Tile` data types, each of which contains 4 `char`s representing that tile's top, left, right, and bottom colors. Since both the code and output terminal are in black and white, the term “colors” here is sometimes a misnomer. The colors internally are represented as ASCII characters 'A' – 'Z', which can be converted into actual colors with a separate, optional visualizer. While I had considered it during development,

in this project, tiles can not be rotated, making the puzzle's rules equivalent to TetraVex, rather than Eternity II. After the input array is produced, the programmatically generated puzzle is discarded and can no longer be accessed. It is now up to the solver to recover a solution. A solution is valid if all the internal edges match, regardless of if it was the generator's intended solution or not.

When generating a puzzle, there are several configuration options that determine its difficulty. The size of the puzzle is the most straightforward setting. As size increases, so does the search space. Since the total number of possible board states, valid or not, applies both a square and a factorial to the size, even "small" puzzles present a challenge. The number of distinct edge colors, referred from now on as "color count", is another configuration setting that affects the difficulty of the puzzle, but it is less straightforward than size. A color count of 1, for example, produces a puzzle that is trivially easy to solve since every configuration is a valid one. A very high color count is also easy to solve, since there would be very few configurations, outside of the valid one, where the pieces could even fit together, leading to a very low branching factor and a very constrained search space. As color count increases, the number of valid solutions decreases, alongside the branching factor for each possible move. The extremes produce easier puzzles, while color counts closer to the middle are more challenging. One final configuration option is whether or not the puzzle is framed or unframed. The Eternity II puzzle is an example of a framed puzzle. Valid solutions to Eternity II have a gray border, and a solution with grey edges not on the border is considered invalid. As a direct result there are 4 pieces that must be on the corners, and $n-2$ pieces that must go on each edge. Regular TetraVex puzzles found in the Microsoft or GNOME versions of the game, by contrast, are unframed. The border of the puzzle is made of colors which can appear elsewhere, though the border colors do not need to be matched themselves. While it is possible to identify border pieces by finding which colors are unmatched on a given side, there are far fewer border pieces that can be identified this way than there are on framed puzzles. As a result, framed puzzles tend to be significantly easier than unframed puzzles. To highlight this, when placing the top left piece, which all of the solvers in this project do first, n pieces can be placed there on an unframed puzzle, compared to only 1 on a framed one, since there is only one piece with the border color on the top and on the left. In the code, n represents the puzzle's physical size, c represents the color count, and f is a boolean representing whether or not the puzzle is framed.

Data Structures

Tiles are, as mentioned previously, represented as a `struct` containing four `chars` representing that tile's color.

The board state, alongside functionality to manipulate it, are contained within a `Board` class. This class contains a representation of the board itself as a two dimensional vector of `Tile*` structures. `Board` provides an interface to interact with the board via `place_tile` and

`get_tile` methods, which take in a row and column on the board as input. Additionally, the `Board` class provides a function to `shuffle` the tiles, used when generating input, and to `validate` a board's solution, which returns true when the puzzle is solved.

A third struct, `SolverState`, contains both an instance of `Board` and a vector of unplaced tiles. This structure provides separation between the already placed pieces and the pieces yet to be placed, while allowing them to travel together. In the backtracking searching algorithms used in the solvers, searching is performed over instances of `SolverState`.

These data structures are shared amongst the serial, shared, and distributed memory solvers.

Serial and Shared Memory Approach

The serial approach is a breadth-first search tree with pruning. At the beginning, the blank solver state with all of the tiles in the unplaced vector is placed into a search queue. Then, in the loop, the first element in the queue is popped. The pruning algorithm chooses the coordinate that has the lowest possible number of tile options available. This ensures that the branching factor is as low as possible. If there is a coordinate with 0 options available, that means the puzzle is unsolvable from that state. The board states with the chosen coordinate filled in are then placed onto the queue, and the loop repeats until the queue is empty. If the vector of tiles to be placed is empty, and calling `validate` on the `Board` class returns true, then we have a valid solution and print it out. This does not stop the search tree from running, so this method will print every possible solution. From my experience, it is common for a set of tiles to yield around 3 or 4 solutions, rather than just one.

The shared memory approach is similar to the serial approach, but it runs multiple threads, sharing some memory among them. Specifically, the queue of `SolverState` is shared. Since the search algorithm is breadth-first, there will usually be thousands of states in the queue, if not more, until the process nears completion. This makes it easy to parallelize using OpenMP. The most time consuming parts of this solver occur once a `SolverState` has been chosen, and, once a `SolverState` has been pulled, calculating constraints and placing new tiles can be done in parallel. In the loop, there are two critical sections: one at the beginning and one at the end. At the beginning of the loop, an element is popped off of the queue and it is stored. During the parallel section, the calculated next states are stored in a temporary vector called `to_place`. Then, at the end of the loop, everything in `to_place` is appended to the queue at once. Doing this, rather than directly placing each new `SolverState` to the queue as soon as it is calculated, minimizes the total number of critical sections found within the loop, ensuring we spend as much time as possible within parallel computing.

The shared memory approach provides helpful speedup, but there are also numerous downsides. Most notably, breadth first search is inefficient when it comes to puzzles with low color counts. If there was only 1 color and every piece was the same, a breadth first search would be noticeably more inefficient than it would be for a puzzle with a higher color count and lower branching factor. Furthermore, on very large puzzle sizes the shared memory approach will run out of space before the puzzle can be solved. For these reasons, I decided to also attempt a solution with distributed memory.

Distributed Memory Approach

Due to the performance downsides of the shared memory approach, I wrote a second version of the solver that uses distributed memory. The distributed memory solver uses MPI, and spreads the work across multiple processes. With distributed memory, there are no longer any possibilities for race conditions. Therefore, there are no critical sections preventing other processes from performing their task while another process accesses memory. The lack of shared memory, however, presents its own challenge: there needs to be some way for the processes to communicate. Crucially, no process should ever consider a board state that has already been considered in a different process.

The distributed memory system begins with a process that splits up work into easy to transmit pieces while ensuring minimal overlap between them. This is accomplished using a short breadth-first search that will halt when the size of the search queue equals or exceeds some factor of the number of processes. These pieces are then transmitted to the other processes, which then run their own search. These searches themselves are depth-first. Using depth-first search keeps the total memory footprint small, since unlike a breadth-first search, elements are quickly added and removed from a depth-first search, and they don't pile up. This piling up is useful when sharing memory, but it is only a problem with distributed memory. Furthermore, a depth-first search is more likely to find a solution earlier, since a board state with x placed pieces is allowed to be checked before another state with $x - 1$ placed pieces in depth-first, but not in breadth-first. Since every solution occurs at the very deepest part of search space, the theoretical advantage of finding the shallowest solution with breadth-first searching does not exist. Unlike the shared solver, this solver will only look for one solution per process. Once a process finds a solution, it sends back its result and terminates.

Before performing any tree searches, the solver needs to convert the board into a form that is easy to transmit between processes. MPI makes it easy to send over arrays of integers, floats, and booleans, but it is more difficult to send over entire structs. Furthermore, assuming every process loads in the initial set of tiles on its own, it isn't necessary to send the entire `SolverState` struct over, since a state's vector of unused tiles can be computed from the input tile list and the tiles placed on the board. While this calculation would be inefficient to perform every time a new

board state is examined during the search loop, using it when transmitting board states across processes reduces the amount of communication required.

Input into the solver can take one of two forms: either a seed for the random puzzle generator, or a one dimensional list of tiles. Either way, the same input is fed into all of the processes. Due to this, every process has the same tile array, and can create the same mappings from MPI friendly integers to corresponding `Tile*`s and back. Tile indices are based on placement in the original vector of tiles array, which is constant based on the input. For example, if a tile has index 0 on one process, then the equivalent tiles on the other processes with the same colors on the same edges also map to index 0. As a result, the processes can easily communicate which tiles to send in a solver state by sending arrays of indices from the original tile set.

After input is generated and the initial work is created via a breadth-first search, it is sent to the other processes. The manager process sends flattened arrays of n by n integers to each of the other processes. These arrays contain placements for pieces based on their calculated index, with -1 representing an empty cell. The worker processes then take these arrays and convert them into valid `SolverState*`s, which are then placed into the depth first search. In order to provide constant expectations between the processes, the same amount of work is sent to each process.

After finding one valid solution, the worker process prints out its board state, the time it took to find the solution, and terminates. If no solution is found, either due to running out of time or running out of work without finding a valid solution, the time is still printed, and the process simply halts.

Load Rebalancing with Distributed Memory

With the distributed memory solution, some processes will run out of work early on, while other processes might take a long time. In the original distributed memory solution, when a process runs out of work, it just stops. As a result, parallelism decreases as time progresses. In some cases, all processes but one could terminate before finding a solution, effectively making the solver serial again.

To remedy this, the distributed memory solver can perform periodic load rebalancing. In some cases, such as those where processes rarely run out of work, load rebalancing would just add overhead. In other cases, processes may frequently run out of work, and without load rebalancing, most of the cores would be wasted. Since there are some scenarios where it is beneficial and some where it is harmful, load rebalancing is optional.

When load rebalancing is on, each worker process will periodically communicate with the manager process after performing some constant amount of work. At that point, each worker will send how much work is currently in its search tree to the manager, and wait for a broadcast. If a

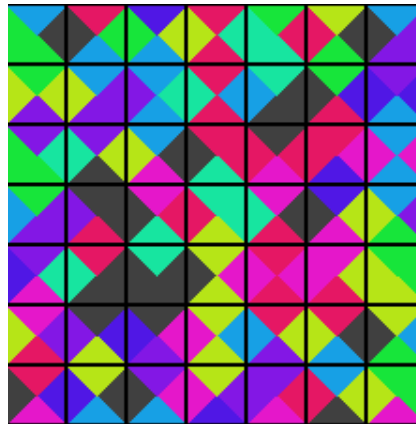
worker finds a solution, it will tell the manager that it has -1 work remaining, which would signal the manager to send a stop command.

After each worker has sent its work total to the manager, the manager will send a broadcast. If a worker has found a solution, the manager broadcasts -1 to all the workers, telling them to stop. If there are no idling workers, the manager broadcasts a signal telling the workers to quit waiting for rebalancing broadcasts and resume working. If there is an empty worker, the manager sends a broadcast containing the ID of the empty worker and the ID of the worker that currently has the most work on its stack. The donor worker then sends work to the empty worker, stopping it from idling. This transfer of work is done peer-to-peer between the two workers with no further input from the manager thread. This continues until there are no more idle workers.

Periodic load rebalancing can be tuned by changing the period p , which represents how many cycles through the loop a process will go before entering load rebalancing. A very low p will rebalance often, ensuring that processes are rarely idling. The downside of low p is an increase in communication overhead. Increasing p will decrease communication overhead at the cost of processes being idle longer. This value should be tuned based on the puzzle, and how often processes run out of work when solving it. Generally, larger puzzles with narrower search spaces, like large framed puzzles with high color counts, benefit most from load rebalancing.

Performance and Analysis

7x7 TetraVex Puzzle with 9 Colors



7x7 TetraVex puzzle with 9 colors, represented visually, solved

The first test case is unframed, with 9 distinct colors and a size of 7x7. It was generated randomly with the numerical seed 0.

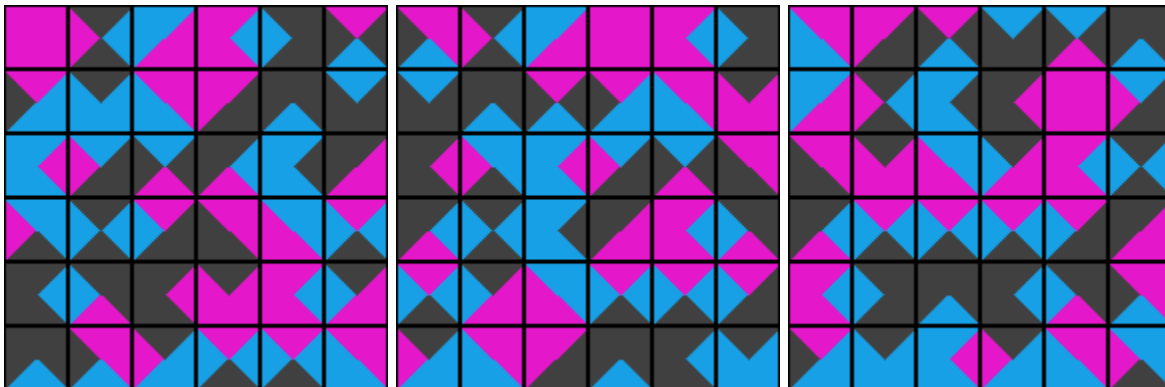
The serial version of the program did not solve the puzzle in time. This was the most complex type of puzzle that my original Python solver could process, and that often took up to an hour to

solve puzzles of this size and color count. Additionally, this puzzle has a greater size than any found in Microsoft TetraVex. Future puzzles are more complex than this, so the serial solver was not able to efficiently solve any of them.

The shared memory approach did solve the puzzle. It took 53.502 seconds with 36 threads running in parallel. The shared memory solver only found one solution for this particular set of tiles. Since the breadth first search does not halt upon finding just one solution, this proves that this particular puzzle only has one solution.

The distributed memory approach with load rebalancing turned off solves the puzzle after 11.1459 seconds with 35 worker threads. This demonstrates, at least for this puzzle, a significant speedup with regards to the shared memory solution. The speedup can be explained by the reduction in branching factor after the first tile is placed. It can also be explained by the lack of any critical section within the distributed memory version after the initial state is sent over. There is no process waiting on any other process to finish its work in a section before another process can continue. Based on these results, there is a significant benefit to solving TetraVex puzzles of this size and complexity using distributed memory parallelization over shared memory. Since this puzzle was solved quickly with distributed memory on its own, load rebalancing was unnecessary.

Puzzle with Very Few Colors



Three different solutions made with the same group of pieces

This test case was made to test the advantage that a distributed depth first approach would have over a shared breadth first approach. In this puzzle, there are only 3 colors, and therefore, there are a large number of potential solutions.

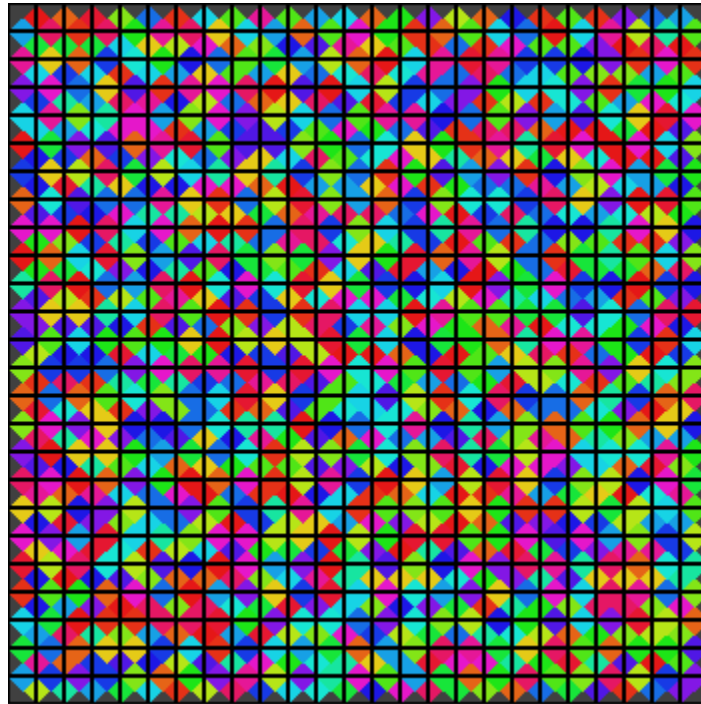
The distributed memory approach without load rebalancing finds its first solution at 0.28 seconds with 35 worker processes, and it ends up finding 10 total solutions within 5 minutes. Load rebalancing is simply unnecessary for a puzzle of this type, especially because the 0.28 seconds

needed to solve the puzzle is lower than the 0.71 seconds needed to finish sending initial states to every process.

The shared memory approach, by contrast, does not find any solution, and runs out of memory before running out of time due to the high branching factor.

These results effectively demonstrate the advantages that combining distributed memory and depth first searching provides. A serial depth first search likely would have taken a lot longer as well, since the work initially distributed to all of the processes emphasizes breadth. This puzzle is 6x6, and it was run with 36 processes, meaning that each possible top-left corner is evaluated on its own process. Since the top left corner of an unframed puzzle has the highest branching factor, the largest bottleneck on a serial depth first search is removed when parallelizing with distributed memory.

Very Large Framed Puzzle with 23 Colors

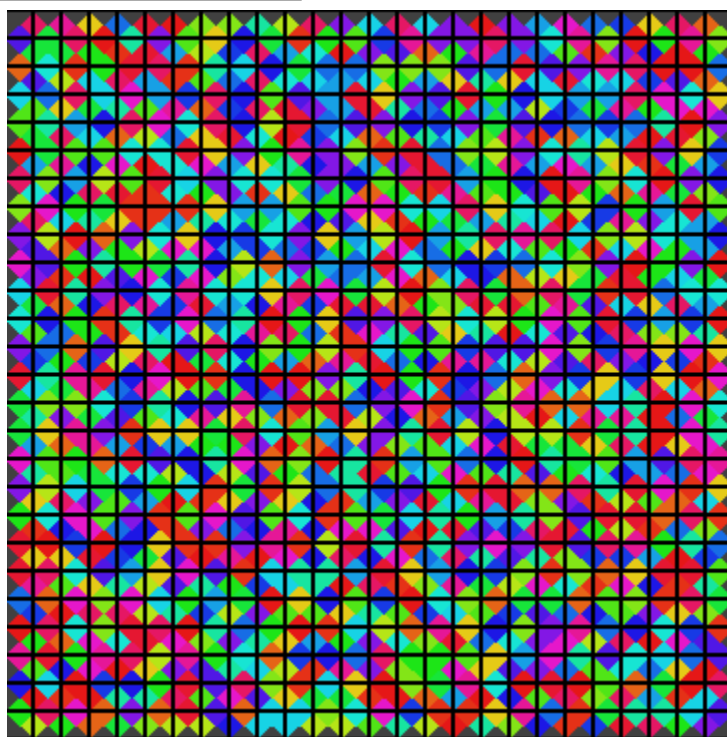


25x25 Framed Solution with 23 Colors

The Eternity II puzzle is 16 by 16, framed, and has 23 colors. Unlike TetraVex, however, Eternity II pieces can be rotated, increasing the branching factor. This solver, therefore, would be able to solve Eternity II if it somehow knew the correct rotations of each piece, which is its own difficult problem.

Unlike the previous two puzzles, the shared memory solver performs better on this one. The shared memory solver can find the solution in only 7.764 seconds, while it takes 73.5356 seconds to find the solution with distributed memory with load rebalancing turned off. When enabling load rebalancing, with p set to 20000, the distributed memory falls to taking only 22.42 seconds. By lowering p to 500, distributed memory takes only 11.66 seconds. In this scenario, load rebalancing provides significant speedup, but in this scenario, shared memory is still the most efficient. This is likely because this specific puzzle benefits from very high, consistent throughput. On systems with tight memory constraints, however, distributed memory with load rebalancing would be the best option, since it performs comparably while having a lower memory footprint.

Even Larger Framed Puzzle with 23 Colors



26x26 Framed Solution with 23 Colors

On the Great Lakes computer, puzzles with a size of $n = 26$ run out of memory with the shared memory solver. At this point, a distributed memory solver is required.

With periodic load rebalancing turned off, this puzzle is not solved in time. Additionally, by the end of 5 minutes, only 4 processes are even running, with 26 terminating within the first minute.

With $p = 20000$, distributed memory with periodic load rebalancing takes 273.00 seconds to solve this puzzle. This solution makes full use of periodic load rebalancing. While running this solution, processes run out of work 469 times. Additionally, every process runs out of work at

least three times, showing that this load rebalancing spreads the work around in an equitable way.

Lowering p to 500 reduces the time to 103.263 seconds. Based on this result and the result of the previous large puzzle, it is better to have more communication overhead so long as process idle time is as minimized. Lowering p below 500, such as to 250, causes the time to increase again. This is due to the rebalancing routine occurring when every process is busy, which accomplishes nothing material except for costing overhead.

Further Work

The most obvious next step is to add piece rotation to the solver. Doing so will increase the potential number of configurations of a puzzle from $(n^2)!$ to $4^{(n^2)}(n^2)!$. Rotations will also increase the branching factor substantially. When only considering position, a tile with red on the top can only match a tile with red on the bottom edge. Adding in rotation, and a tile with red on any edge can match any other tile with red on any edge. This increases the search space significantly, and further pruning algorithms would likely be necessary to solve puzzles of a similar size to those solved here.

Additionally, more complex load rebalancing algorithms would likely further reduce time spent solving TetraVex puzzles. One suggestion would be dynamically changing p based on how many processes are actually busy at any given time, increasing it if load rebalancing doesn't occur and decreasing it if processes idle for too long.

Finally, the process of tuning p itself might be an interesting problem for the future. At present, I made incremental changes to it and checked if that sped up the solver or not. It might be possible to further optimize p using machine learning. Additionally, it might be interesting to figure out whether optimal p is tied to a specific puzzle, or if there is one specific setting for p that works well in all cases. Based on my experimentation, I think that there is likely a certain range of optimal p , likely within 250 and 2000, but the specific value is likely puzzle-specific, or even changes throughout the solution process.

Conclusion

TetraVex, depending on the configuration based on board size, color count, and whether or not it is framed, is a difficult puzzle to solve. Serial algorithms are not sufficient to solve large TetraVex puzzles, making parallelization helpful in solving them quickly within a short timeframe with finite memory.

TetraVex puzzles can be solved with both shared and distributed memory approaches. Based on the four benchmark puzzles, some configurations are easier to solve with shared memory, and some are easier with distributed. Puzzles that have a larger solution space where distributed

processes starting the puzzle from different start states often find different solutions are more easily solved with distributed memory. At the same time, puzzles with a very large search space but few solutions, which benefit from a constant stream of work rather than some processes ending and some finding solutions, benefit from shared memory. By rebalancing work from busy processes to idle processes, the distributed memory solver can solve puzzles normally suited for shared memory, while generally improving on the performance of the distributed memory solver. The method used for solving a TetraVex puzzle should be based on that puzzle's configuration, but distributed memory with load rebalancing provides both high performance and a low memory footprint, enabling it to solve larger puzzles than shared memory.

References

[1] <https://www.sciencedirect.com/science/article/abs/pii/S002001900600130X?via%3Dihub>

[2]

https://e2.bucas.name/#puzzle=Joshua_Blackwood_470&board_w=16&board_h=16&board_edges=ajraaftjabvfafpbajefarcjablrajubafpjanefajmnabpjafebabifafbsaabfruratiouvvsiptuvewstcugwlwouuicwpdlieiidmocipctoolecigoescgbabcrifaootisddoudmdsggdlddgopddcluplmwlimemicswttpcehutotvhcwetbajvftratuttdchumstcgvvsvdwvvdwvuwewwluhedvlsmsdptemupwtvvdcepemjarprijatdoihltdtmolvwomvcwwvcieesscuhssvtihshltetehwdltdhcdeighrarcjhjaoelh tileoliosclwcssitpcseptsicilmclshlehhs lgvhcwvgppvwrafpjmbalhmml luhimulcmimsggmpppgpmhpiovmmw pphsewhwqsvwiwvovwvqdofafqgbamhtguqghucmqitecglstpeclhmsevlompgtlehvgqtohievtvpmedutpfa fubhfatplhgiipmqgiemdqsdpmlidsgglodegtohdvohoocsovcucmgotikgfafiflnalqpliluqghvlddw hpdqdikgdgepkewiehdwhccosugcuouoegukwlefarwnqnapdhqutkdvkqtwedkqswegpgspuopi hpudqchcueqgqkuousqgpmulswprarsncnahqvckwoqqwuwdmewwvdmgttvovmtpkivcs lkepkskuups qgumsgqwsdsrajsnhfavqwhogkqukvgeowkdesothqemklhimwklklmkqt kuikqgsuigtvsdvktjanvfdnawlqdkgqlvlqgwqklsuhqqvqulimvwkpiloekt kuokdckuqkdvpukompnajokngbaqhggqm khqkkmkwgkhoqwqv iomokvpmkoeqtmumiqcdhmkqedukwqmpkkjajpbraaqnarkjankbajgnabqbaninabkbankjabtnajiranhnarenanwrankrarjaar&motifs_order=jblackwood

[3] https://groups.io/g/eternity2/topic/another_470/109886274